DIGITAL & LOGIC CIRCUITS

Design Document: Two-Road Traffic-Light Controller and MOD-6 Synchronous Counter

Student name: janhavi chaudhari **Roll number:** 46

Course: Digital Electronics / Digital and Logic Circuits **Date:** 10/11/26

Abstract

This document presents two sequential-logic designs. The first is a two-road traffic-light controller implemented as a Moore finite-state machine (FSM). The second is a MOD-6 synchronous binary counter. Both designs use three D flip-flops, a common clock, state-transition logic, and Boolean simplification using Karnaugh maps (K-maps).

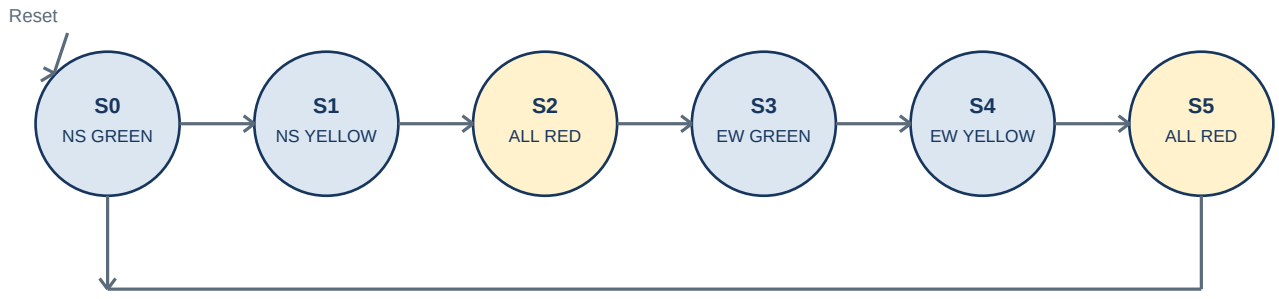
Design assumptions

• The roads are called North–South (NS) and East–West (EW).
• The controller advances one state on each active clock edge. Actual light durations can be set by a slower enable pulse from a timer/divider.
• The two roads are never given green at the same time. An all-red clearance state is inserted between directions.
• State encoding is binary: S0=000, S1=001, S2=010, S3=011, S4=100, S5=101. States 110 and 111 are unused and treated as don't-cares during minimisation; reset forces the machine to 000.
• The counter counts 0, 1, 2, 3, 4, 5, then returns to 0.

1. Two-road traffic-light controller (Moore FSM)

A Moore machine is selected because each light output depends only on the present state. This makes the output pattern stable throughout a state and independent of changes in external inputs between clock edges.

1.1 State sequence and meaning



One clock pulse per state transition; reset starts at S0

State	Code Q2Q1Q0	NS signal	EW signal	Next state
S0	000	Green	Red	S1
S1	001	Yellow	Red	S2
S2	010	Red	Red (clearance)	S3
S3	011	Red	Green	S4
S4	100	Red	Yellow	S5
S5	101	Red (clearance)	Red	S0

1.2 State-transition table and D-input values

For D flip-flops, the next-state bits are applied directly to the D inputs: D2D1D0 = next state. The state sequence is cyclic.

Present state	Q2Q1Q0	Next state	D2D1D0
S0	000	001	001
S1	001	010	010
S2	010	011	011
S3	011	100	100
S4	100	101	101
S5	101	000	000
Unused	110 / 111	Don't care	XXX

1.3 K-map minimisation for next-state logic

Use Q2 as the row variable and Q1Q0 as the column variables in Gray-code order 00, 01, 11, 10. Minterms 6 and 7 are marked X because those state codes are unused.

D2	Q1Q0 = 00	01	11	10
Q2 = 0	0	0	1	0
Q2 = 1	1	0	X	X

D2 = Q1Q0 + Q2Q0′

D1	Q1Q0 = 00	01	11	10
Q2 = 0	0	1	0	1
Q2 = 1	0	0	X	X

D1 = Q1Q0′ + Q0Q1′Q2′

D0	Q1Q0 = 00	01	11	10
Q2 = 0	1	0	0	1
Q2 = 1	1	0	X	X

D0 = Q0′

The simplified equations above drive the three D flip-flops. A common clock advances the state sequence. A reset input should initialise the flip-flops to 000 (S0).

1.4 Output table and output equations

State	NS_G	NS_Y	NS_R	EW_G	EW_Y	EW_R
S0 (000)	1	0	0	0	0	1
S1 (001)	0	1	0	0	0	1
S2 (010)	0	0	1	0	0	1
S3 (011)	0	0	1	1	0	0
S4 (100)	0	0	1	0	1	0
S5 (101)	0	0	1	0	0	1

Output names: NS_G / NS_Y / NS_R are North–South green/yellow/red; EW_G / EW_Y / EW_R are East–West green/yellow/red. A value of 1 means that lamp is ON.

Minimised output equations (using states 110 and 111 as don't-cares):

NS_G = Q2′Q1′Q0′
NS_Y = Q2′Q1′Q0
NS_R = Q1 + Q2
EW_G = Q1Q0
EW_Y = Q2Q0′
EW_R = Q0Q1′ + Q0′Q2′

Output K-maps

The following maps use the same variable order as the next-state maps. They show the output minterms and the unused-state don't-cares.

NS_G	Q1Q0 = 00	01	11	10
Q2 = 0	1	0	0	0

Q2 = 1	0	0	X	X

NS_Y	**Q1Q0 = 00**	**01**	**11**	**10**
Q2 = 0	0	1	0	0
Q2 = 1	0	0	X	X

NS_R	**Q1Q0 = 00**	**01**	**11**	**10**
Q2 = 0	0	0	1	1
Q2 = 1	1	1	X	X

1.4 Output K-maps (continued)

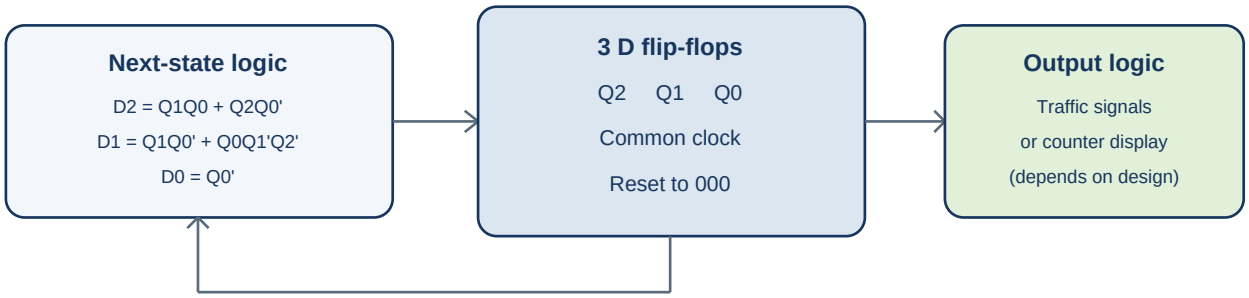
EW_G	Q1Q0 = 00	01	11	10
Q2 = 0	0	0	1	0
Q2 = 1	0	0	X	X

EW_Y	Q1Q0 = 00	01	11	10
Q2 = 0	0	0	0	0
Q2 = 1	1	0	X	X

EW_R	Q1Q0 = 00	01	11	10
Q2 = 0	1	1	0	1
Q2 = 1	0	1	X	X

1.5 Logic circuit implementation

Connect three edge-triggered D flip-flops to a shared clock. Feed the D inputs with the equations derived above. Connect Q2, Q1 and Q0 to the output decoder that implements the six lamp equations. Provide a reset that sets Q2Q1Q0=000. In a practical model, use a clock divider or timer to create a suitable interval between traffic states; the state machine itself only defines the order of states.



Implementation checklist

- Use three D flip-flops with a common clock.
- Apply D2, D1 and D0 equations to the corresponding D pins.
- Decode the six lamp outputs using the output equations.
- Initialise to S0 using reset.
- Verify that NS and EW green outputs are never simultaneously 1.
- Verify the all-red clearance states S2 and S5.

2. MOD-6 synchronous counter

A MOD-6 counter has six valid states. It counts upward in binary from 000 to 101 and then wraps around to 000. Three flip-flops are required because $2^2 < 6 \leq 2^3$.

2.1 Counting sequence

Clock count	Present output Q2Q1Q0	Next output Q2■Q1■Q0■
0	000	001
1	001	010
2	010	011
3	011	100
4	100	101
5	101	000

2.2 State-transition table

Q2	Q1	Q0	D2	D1	D0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	0	0	0
1	1	0	X	X	X
1	1	1	X	X	X

2.3 K-maps and simplified equations

The MOD-6 counter has the same six-state transition sequence and binary state assignment as the traffic-light FSM. Therefore its next-state K-maps and equations are the same.

D2	Q1Q0 = 00	01	11	10
Q2 = 0	0	0	1	0
Q2 = 1	1	0	X	X

D2 = Q1Q0 + Q2Q0′

D1	Q1Q0 = 00	01	11	10
Q2 = 0	0	1	0	1
Q2 = 1	0	0	X	X

D1 = Q1Q0′ + Q0Q1′Q2′

D0	Q1Q0 = 00	01	11	10
Q2 = 0	1	0	0	1
Q2 = 1	1	0	X	X

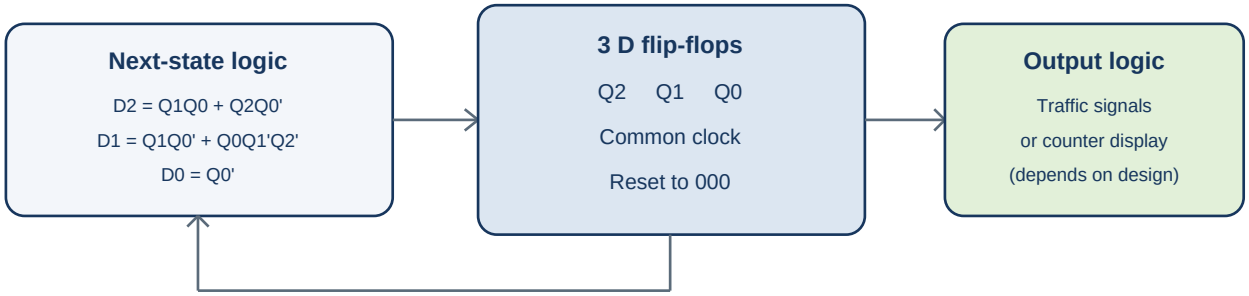
D0 = Q0′

2.4 Wrap-around logic

The required wrap-around occurs when the present count is 101 (decimal 5). On the next active clock edge, the next-state logic sets the flip-flops to 000. This is a synchronous transition: the counter does not need a separate asynchronous clear at count 5. The reset input is used only to initialise the counter.

2.5 Counter circuit

Use three D flip-flops connected to the same clock. Connect their D inputs to the three simplified equations. The Q outputs provide the 3-bit count. An optional decoder can convert the binary count to a display format, but it is not required for the MOD-6 counting function.



3. Verification and discussion

3.1 Functional verification

Test	Expected result
Traffic FSM reset	State becomes 000 (NS green, EW red).
Traffic FSM sequence	000 → 001 → 010 → 011 → 100 → 101 → 000.
Green-light interlock	NS_G and EW_G are never ON together.
Clearance	Both roads show red in states 010 and 101.
Counter sequence	000, 001, 010, 011, 100, 101, then 000.
Counter modulus	Exactly six distinct count states occur before repeat.
Reset	Both designs start at 000 when reset is asserted.

3.2 Why Moore was chosen instead of Mealy

A Moore FSM was chosen for the traffic-light controller because its outputs depend only on the current state. Therefore, the red, yellow and green lamp combinations remain stable for the full duration of each state and do not change immediately in response to an input fluctuation. This makes the controller easier to understand, test and verify. A Mealy machine can sometimes use fewer states because its outputs depend on both the state and inputs, but input changes can affect outputs between clock edges. For a simple traffic-light sequence, the predictable state-based outputs of a Moore machine are a useful design choice.

3.3 Design notes and limitations

This is an educational logic design, not a certified road-traffic controller. A real installation would require appropriate sensors, pedestrian handling, fault detection, timing standards, fail-safe hardware and independent safety review. The simple design assumes a fixed repeating sequence and uses a clock-enable interval to set the duration of each phase.

4. Conclusion

The traffic-light controller and MOD-6 counter have been designed using three-bit state encoding and D flip-flops. K-map minimisation produces the next-state equations and traffic-light output equations. The traffic controller uses six Moore states, including two all-red clearance states, while the counter cycles through six binary values and synchronously returns to zero after count five.

Submission reminder: Review the assumptions and equations, add your name and student details, and export/upload this PDF to the GitHub submission link provided by your course. If your instructor specifies different road timing or a different counter modulus, update the design accordingly.